DATA STRUCTURES
LEVEL 1

1. A list of mail addresses represented as integers are stored in “address_array[]”, which are not in any specific order. You need to sort this list using the bubble sort mechanism.

- You start at the beginning of the address_array[] and compare each address with the next one.
- If the current address value is greater than the next address, you swap them.
- Continue this process until no more swaps are needed, meaning the array is sorted in non-decreasing order.
- The final sorted list should be stored in Sort_address_array[].

Let's denote the following variable,

Address_array[] → An input integer array of length n
Sort_address_array[] → An output integer array of length n

Example 1:
Input:
Enter the number of input mail addresses: 5
Enter the address values: 9 5 1 4 2
Output:
Sorted address for delivery: 1 2 4 5 9

Example 2:
Input:
Enter the number of input mail addresses: 6
Enter the address values: 7 7 7 7 7 7
Output:
Sorted address for delivery: 7 7 7 7 7 7

Constraints:

- 1<= n <10
- The elements in the address_array[] and Sort_address_array[] are positive integers.

Test Case 1
Input:
Number of addresses: 5
Addresses: 9 5 1 4 2
Expected Output:
Sorted addresses: 1 2 4 5 9

Test Case 2
Input:
Number of addresses: 6
Addresses: -5 3 -1 2 -8 4
Expected Output:
Sorted addresses: -8 -5 -1 2 3 4
Test Case 3
Input:
Number of addresses: 5
Addresses: -9 -5 -1 -4 -2
Expected Output:
Sorted addresses: -9 -5 -4 -2 -1
Test Case 4
Input:
Number of addresses: 5
Addresses: -9000 -5000 -1000 -4000 -2000
Expected Output:
Sorted addresses: -9000 -5000 -4000 -2000 -1000
Test Case 5
Input:
Number of addresses: 4
Addresses: 3000 -1000 4000 -2000
Expected Output:
Sorted addresses: -2000 -1000 3000 4000

2. Given an array “nums” with “n” objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue. Solve this problem using selection sort technique without using the predefined sort function

Constraints:

- nums[] → An integer array of colored objects [color representation (red - 0, white -1, blue -2)]
- n = nums.length

Example 1

Input: [2, 0, 2, 1, 1, 0]

Explanation:

1. Initial Array: [2, 0, 2, 1, 1, 0]
2. First Pass (Sorting 0s):
    - Traverse the array to find 0s.
    - Swap each found 0 with the current position pointed to by start.
    - start is incremented each time a 0 is placed correctly.
    - After first pass, nums becomes [0, 0, 2, 1, 1, 2] with 0s in place.
3. Second Pass (Sorting 1s):
    - Traverse the array from where the 0s end to find 1s.
    - Swap each found 1 with the current position pointed to by mid.
    - mid is incremented each time a 1 is placed correctly.
    - After second pass, nums becomes [0, 0, 1, 1, 2, 2] with 1s in place.
4. Final Result: [0, 0, 1, 1, 2, 2]

Example 2

Input: [1, 2, 0, 0, 1, 2]

Explanation:

1. Initial Array: [1, 2, 0, 0, 1, 2]
2. First Pass (Sorting 0s):
    - Traverse the array to find 0s.
    - Swap each found 0 with the current position pointed to by start.
    - start is incremented each time a 0 is placed correctly.
    - After the first pass, nums becomes [0, 0, 2, 1, 1, 2] with 0s in place.
3. Second Pass (Sorting 1s):
    - Traverse the array from where the 0s end to find 1s.
    - Swap each found 1 with the current position pointed to by mid.
    - mid is incremented each time a 1 is placed correctly.
    - After the second pass, nums becomes [0, 0, 1, 1, 2, 2] with 1s in place.
4. Final Result: [0, 0, 1, 1, 2, 2]

Test Case 1
Input: [2, 0, 2, 1, 1, 0]
Explanation:

1. Initial Array: [2, 0, 2, 1, 1, 0]
2. Sort 0s:
    - First 0 found at index 1. Swap it with the element at index 0: [0, 2, 2, 1, 1, 0].
    - Next 0 found at index 5. Swap it with the element at index 1: [0, 0, 2, 1, 1, 2].
3. Sort 1s:
    - First 1 found at index 3. Swap it with the element at index 2: [0, 0, 1, 2, 1, 2].
    - Next 1 found at index 4. Swap it with the element at index 3: [0, 0, 1, 1, 2, 2].
4. Final Result: [0, 0, 1, 1, 2, 2]

Test Case 2
Input: [1, 2, 0, 0, 1, 2]
Explanation:

1. Initial Array: [1, 2, 0, 0, 1, 2]
2. Sort 0s:
    - First 0 found at index 2. Swap it with the element at index 0: [0, 2, 1, 0, 1, 2].
    - Next 0 found at index 3. Swap it with the element at index 1: [0, 0, 1, 2, 1, 2].
3. Sort 1s:
    - First 1 found at index 2. No swap needed, but we check the next 1s.
    - Second 1 found at index 4. Swap it with the element at index 2: [0, 0, 1, 1, 2, 2].
4. Final Result: [0, 0, 1, 1, 2, 2]

Test Case 3
Input: [0, 1, 2, 2, 1, 0]
Explanation:

1. Initial Array: [0, 1, 2, 2, 1, 0]
2. Sort 0s:
    - First 0 found at index 0, so no change.
    - Next 0 found at index 5. Swap it with the element at index 1: [0, 0, 2, 2, 1, 1].
3. Sort 1s:
    - First 1 found at index 4. Swap it with the element at index 2: [0, 0, 1, 2, 2, 1].
    - Next 1 found at index 5. Swap it with the element at index 3: [0, 0, 1, 1, 2, 2].
4. Final Result: [0, 0, 1, 1, 2, 2]

Test Case 4
Input: [2, 1, 0, 1, 0, 2]
Explanation:

1. Initial Array: [2, 1, 0, 1, 0, 2]
2. Sort 0s:
    - First 0 found at index 2. Swap it with the element at index 0: [0, 1, 2, 1, 0, 2].
    - Next 0 found at index 4. Swap it with the element at index 1: [0, 0, 2, 1, 1, 2].

3. Sort 1s:
    - First 1 found at index 3. Swap it with the element at index 2: [0, 0, 1, 2, 1, 2].
    - Next 1 found at index 4. Swap it with the element at index 3: [0, 0, 1, 1, 2, 2].
4. Final Result: [0, 0, 1, 1, 2, 2]

Test Case 5

Input: [1, 2, 2, 0, 0, 1]

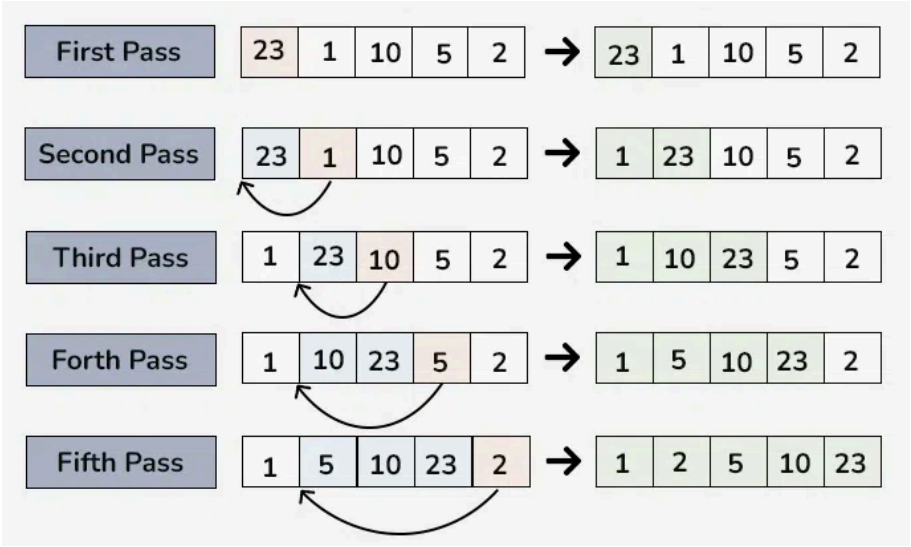
Explanation:

1. Initial Array: [1, 2, 2, 0, 0, 1]
2. Sort 0s:
    - First 0 found at index 3. Swap it with the element at index 0: [0, 2, 2, 1, 0, 1].
    - Next 0 found at index 4. Swap it with the element at index 1: [0, 0, 2, 1, 2, 1].
3. Sort 1s:
    - First 1 found at index 3. Swap it with the element at index 2: [0, 0, 1, 2, 2, 1].
    - Next 1 found at index 5. Swap it with the element at index 3: [0, 0, 1, 1, 2, 2].
4. Final Result: [0, 0, 1, 1, 2, 2]

3. Imagine you have a deck of “n” cards, each with a unique number. You want to sort the cards in ascending order using a simple comparison and swap process as given below,

- We have to start with the second element of the array as the first element in the array is assumed to be sorted.
- Compare the second element with the first element and check if the second element is smaller then swap them.
- Move to the third element and compare it with the second element, then the first element and swap as necessary to put it in the correct position among the first three elements.
- Continue this process, comparing each element with the ones before it and swapping as needed to place it in the correct position among the sorted elements.
- Repeat until the entire array is sorted.

Example:
Input: 5
Elements: 23, 1, 10, 5, 2
Explanation:



Output: 1 2 5 10 23

Test case 1: Input: 5 (number of elements)
Elements: -3 -1 -4 -2 -5
Expected Sorted Output: -5 -4 -3 -2 -1

Test case 2 :Input: 6 (number of elements)
Elements: 3 -1 4 -2 0 -5
Expected Sorted Output: -5 -2 -1 0 3 4

Test case 3 Input: 4 (number of elements)
Elements: 0 2 -3 1
Expected Sorted Output: -3 0 1 2

Test case 4 Input: Number of Elements: 5
Elements: 12 7 9 15 4
Expected Sorted Output: 4 7 9 12 15

Test case 5 Input: Number of Elements: 6
Elements: -8 -5 -2 -7 0 -1
Expected Sorted Output: -8 -7 -5 -2 -1

4. Assume the transaction amounts in a large dataset need to be sorted by a data analyst [N indicates the number of transactions]. Once the dataset is sorted, the transactions should be ordered from the smallest to the largest amount [transactions[] indicates the transaction amounts]. Use merge sort to handle the sorting efficiently.

Example 1:

Divide:

[38, 27, 43, 10] is divided into [38, 27] and [43, 10] .

[38, 27] is divided into [38] and [27] .

[43, 10] is divided into [43] and [10] .

Conquer:

[38] is already sorted.

[27] is already sorted.

[43] is already sorted.

[10] is already sorted.

Merge:

Merge [38] and [27] to get [27, 38] .

Merge [43] and [10] to get [10,43] .

Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]

Therefore, the sorted list is [10, 27, 38, 43] .

Example 2

Input:

[41, 22, 17, 59, 10, 33]

Divide:

1. **First Split:**
   - Divide [41, 22, 17, 59, 10, 33] into [41, 22, 17] and [59, 10, 33].
2. **Second Split:**
   - Divide [41, 22, 17] into [41] and [22, 17].
   - Divide [59, 10, 33] into [59] and [10, 33].
3. **Third Split:**
   - Divide [22, 17] into [22] and [17].
   - Divide [10, 33] into [10] and [33].

Conquer:

1. **Base Cases (Single Elements):**
   - [41] is already sorted.
   - [22] is already sorted.
   - [17] is already sorted.
   - [59] is already sorted.

        - [10] is already sorted.
        - [33] is already sorted.
    2. Merge [22] and [17] to get [17, 22].
    3. Merge [10] and [33] to get [10, 33].
    4. Merge [41] and [17, 22]:
        - Merge [41] and [17, 22]:
            - Compare 41 with 17 and 22.
            - 17 and 22 are less than 41, so merge them: [17, 22] then add 41 to get [17, 22, 41].
    5. Merge [59] and [10, 33]:
        - Merge [59] and [10, 33]:
            - 10 and 33 are less than 59, so merge them: [10, 33] then add 59 to get [10, 33, 59].

Final Merge:

- Merge [17, 22, 41] and [10, 33, 59]:
    - Compare elements from both lists:
        - 10 < 17, so add 10 to the result.
        - 17 < 33, so add 17 to the result.
        - 22 < 33, so add 22 to the result.
        - 33 < 41, so add 33 to the result.
        - Add remaining elements 41 and 59 to the result.

Final Sorted List:
[10, 17, 22, 33, 41, 59]

Constraints:
1 <= N <= 1000
1 <= transactions[i] <= 10^7

Test Case 1
Input: [38, 27, 43, 10]
Expected Output: [10, 27, 38, 43]
Test Case 2
Input: [15, 3, 22, 8, 11, 6, 42]
Expected Output: [3, 6, 8, 11, 15, 22, 42]
Test Case 3
Input: [1000, 200, 5000, 150, 3000, 4500, 800]
Expected Output: [150, 200, 800, 1000, 3000, 4500, 5000]
Test Case 4
Input: [9, 7, 5, 3, 1]

Expected Output: [1, 3, 5, 7, 9]

Test Case 5

Input: [55, 23, 89, 12, 34, 76, 42, 10, 7]

Expected Output: [7, 10, 12, 23, 34, 42, 55, 76, 89]

5. Assume the height of all the soldiers in the parade are to be maintained by the captain of the team [N indicates the No of soldiers in the team]. Once the team is appearing for the parade, the captain marks the position of the soldiers based on their heights from shortest to the tallest [height[] indicates the height of the soldier in the team]. Consider the first soldier's height as the pivot such that all the soldiers less than(or equal to) the pivot lie before it and the elements greater than it lie after the pivot. Repeat the same process till the soldiers are placed in their respective positions from shortest to the tallest.

Constraints:

1 <= N <= 100

1 <= height[i] <= 100

Example 1:

Enter the number of soldiers: 5

Enter the heights of the soldiers: 3 1 4 2 5

1. Initial Array: [3, 1, 4, 2, 5]
    - Partition around pivot 3 results in: [2, 1, 3, 4, 5]
2. Left Partition: [2, 1]
    - Partition around pivot 2 results in: [1, 2]
3. Right Partition: [4, 5]
    - Already sorted, no further action needed

Final Sorted Array: [1, 2, 3, 4, 5]

Output:

Sorted heights: 1 2 3 4 5

Example 2:

Enter the number of soldiers: 4

Enter the heights of the soldiers: 38 27 43 10

1. Initial Array: [38, 27, 43, 10]
    - Partition around pivot 38 results in: [10, 27, 38, 43]
2. Left Partition: [10, 27]
    - Already sorted, no further action needed
3. Right Partition: [43]
    - Already sorted, no further action needed

Final Sorted Array: [10, 27, 38, 43]

Output:

Sorted heights: 10 27 38 43

TEST CASE 1:

Enter the number of soldiers: 5

Enter the heights of the soldiers: 1 1 1 1 1

Expected Output:
Sorted heights: 1 1 1 1 1

TEST CASE 2:

Enter the number of soldiers:-1
Expected Output:Number of soldiers must be between 1 and 100

TEST CASE 3:

Enter the number of soldiers:101
Expected Output:Number of soldiers must be between 1 and 100

TEST CASE 4:

Enter the number of soldiers: 5
Enter the heights of the soldiers: 100 100 100 100 100
Expected Output:
Sorted heights: 100 100 100 100 100

TEST CASE 5:

Enter the number of soldiers: 5
Enter the heights of the soldiers: 1 1 1 1 1
Expected Output:
Sorted heights: 1 1 1 1 1

6. An organization maintains an employee database where each employee record consists of an employee's name and age (without using structures concept). Implement a sorting mechanism that arranges these records in ascending order based on the employees' ages using the heap sort algorithm.

Input:

- The first line of input will contain a single integer “n”, denoting the number of employee records.
- The following n lines will each contain two elements: a string representing the employee's name and an integer representing the employee's age.

Output:

- Print the sorted employee records in ascending order based on their ages. Each line should contain the name followed by the age, separated by a space.

Example 1:

Input: 4
Alice 32
Bob 28
Charlie 25
David 30
Output:
Charlie 25
Bob 28
David 30
Alice 32

Example 2:

Input: 5
401 80
402 60
403 40
404 75
405 90
Output:
403 40
402 60
404 75
401 80
405 90

Constraints:

1 <= n <= 10

Employee_name → string

Employee_age → A positive integer

Test Case 1: All Employees Same Age

Input:

3

Alice 25

Bob 25

Charlie 25

Output:

Alice 25

Bob 25

Charlie 25

Test Case 2: Single Employee

Input:

1

Alice 30

Output:

Alice 30

Test Case 3: Two Employees with Different Ages

Input:

2

Bob 45

Alice 30

Output:

Alice 30

Bob 45

Test Case 4: Employees with Ages in Random Order

Input:

4

Zara 29

Liam 34

Nina 25

Emma 31

Output:

Nina 25

Zara 29

Emma 31

Liam 34

Test Case 5: Employees with Large Age Differences

Input:

3

John 100

Anna 20

Mark 50

Output:

Anna 20

Mark 50

John 100

Test Case 6: Ages Already Sorted

Input:

5

Diana 22

Eve 24

Frank 26

Grace 28

Hank 30

Output:

Diana 22

Eve 24

Frank 26

Grace 28

Hank 30

Test Case 7: Input with Varied Length Names

Input:

4

Anna 25

Christopher 22

Zoe 27

Ben 30

Output:

Christopher 22

Anna 25

Zoe 27

Ben 30

Test Case 8: Ages in Decreasing Order

Input:

3

Alice 40

Bob 30

Charlie 20

Output:

Charlie 20

Bob 30

Alice 40

7. A company has a list of employee IDs, they need to quickly find if a particular employee ID exists in the list or not. The list contains unsorted employee IDs. Implement a linear search algorithm to help the company to identify whether a given employee ID is present in the list or not.

Let's denote the following variable,

n → Length of the input array
Emp_ID[] → Input integer array of length n
searchID → Employee ID to be searched in the input array Emp_ID

Example 1:
Input:
Enter number of employee IDs: 5
Enter the employee IDs: 1 2 3 4 5
Enter the employee ID to search: 3
Output:
Employee ID 3 found at index 2

Example 2:
Input:
Enter number of employee IDs: 4
Enter the employee IDs: 10 20 30 40
Enter the employee ID to search: 25
Output:
Employee ID 25 is not found

Constraints:

$1 \leq n \leq 20$
0<=Emp_ID[i]<=10^4
0<=searchID<=10^4

Test case 1:
Input:
Enter number of employee IDs: 5
Enter the employee IDs: 1 2 3 4 5
Enter the employee ID to search: 3
Output:
Employee ID 3 found at index 2

Test Case 2:
Input:
Enter number of employee IDs: 4
Enter the employee IDs: 10 20 30 40
Enter the employee ID to search: 25
Output:
Employee ID 25 is not found
Test Case 3:
Input:
Enter number of employee IDs: 1
Enter the employee IDs: 7
Enter the employee ID to search: 7
Output:
Employee ID 7 found at index 0
Test Case 4:
Input:
Enter number of employee IDs: 1
Enter the employee IDs: 8
Enter the employee ID to search: 5
Output:
Employee ID 5 is not found
Test case 5:
Input:
Enter number of employee IDs: 6
Enter the employee IDs: 4 4 4 4 4 4
Enter the employee ID to search: 4
Output:
Employee ID 4 found at index 0
Test case 6:
Input:
Enter number of employee IDs: 5
Enter the employee IDs: 6 6 6 6 6
Enter the employee ID to search: 8
Output:
Employee ID 8 is not found
Test case 7:
Input:
Enter number of employee IDs: 10
Enter the employee IDs: 15 22 33 44 55 66 77 88 99 100
Enter the employee ID to search: 77

Output:
Employee ID 77 found at index 6

Test case 8:

Input:
Enter number of employee IDs: 10
Enter the employee IDs: 10 20 30 40 50 60 70 80 90 100
Enter the employee ID to search: 55
Output:
Employee ID 55 is not found

Test case 9:

Input:
Enter number of employee IDs: 8
Enter the employee IDs: 5 10 15 10 20 25 10 30
Enter the employee ID to search: 1
Output:
Employee ID 10 found at index 6

Test case 10:

Input:
Enter number of employee IDs: 4
Enter the employee IDs: 0 0 0 0
Enter the employee ID to search: 0
Output:
Employee ID 0 found at index 0

8. A library maintains a catalog of books where each book is identified by a unique ISBN number. The catalog is stored in a sorted array based on the ISBN numbers, allowing for efficient searches. You need to find a specific book in the catalog by its ISBN number. Since the catalog is sorted, Binary Search is an appropriate algorithm to quickly locate the book.

Let's denote the following variable,

n → Length of the input array
books[] → Sorted input integer array of length n
target_ISBN → ISBN number to be searched in the array books

Example 1:
Input:
Enter number of books: 5
Enter the ISBN IDs: 10 20 30 40 50
Enter the target ISBN to search: 20
Output:
ISBN_ID 20 found at index 1

Example 2:
Input:
Enter number of books: 6
Enter the ISBN IDs: 11 22 33 44 55 66
Enter the target ISBN to search: 45
Output:
ISBN_ID 45 is not found

Constraints:

1 <= n <= 20
0<=books[i]<=10^4
0<=target_ISBN<=10^4
Input book array should be sorted in non-decreasing order

Test Case 1:
Input:
Enter number of books: 6
Enter the ISBN IDs: 13 15 17 18 19 22
Enter the target ISBN to search: 22
Output:
ISBN_ID 22 found at index 5

Test Case 2:
Input:
Enter number of books: 5
Enter the ISBN IDs: 33 44 55 57 80
Enter the target ISBN to search: 44
Output:
ISBN_ID 44 found at index 1
Test Case 3:
Enter number of books: 5
Enter the ISBN IDs: -15 -13 -11 -10 -1
Enter the target ISBN to search: -15
Output:
ISBN_ID -15 found at index 0
Test Case 4:
Input:
Enter number of books: 3
Enter the ISBN IDs: 87 98 99
Enter the target ISBN to search: 22
Output:
ISBN_ID 22 is not found
Test Case 5:
Enter number of books: 6
Enter the ISBN IDs: 89 91 96 97 99 101
Enter the target ISBN to search: 100
Output:
ISBN_ID 100 is not found
Test Case 6:
Input:
Enter number of books: 5
Enter the ISBN IDs: 10 33 46 65 67
Enter the target ISBN to search: 65
Output:
ISBN_ID 65 found at index 3
Test Case 7:
Input:
Enter number of books: 4
Enter the ISBN IDs: 21 22 34 45 55
Enter the target ISBN to search: 22
Output:
ISBN_ID 22 found at index 1

Test Case 8:
Enter number of books: 5
Enter the ISBN IDs: -20 3 62 74 88
Enter the target ISBN to search: 77
Output:
ISBN_ID 77 is not found
Test Case 9:
Enter number of books: 4
Enter the ISBN IDs: -30 -20 -15 -10
Enter the target ISBN to search: -15
Output:
ISBN_ID -15 found at index 2
Test Case 10:
Input:
Enter number of books: 6
Enter the ISBN IDs: -1 0 34 45 66
Enter the target ISBN to search: 66
Output:
ISBN_ID 66 found at index 4

9. Given a 2D array of characters grid of size m x n, you need to find if there exists any cycle consisting of the same value in the grid.
A cycle is a path of length 4 or more in the grid that starts and ends at the same cell. From a given cell, you can move to one of the cells adjacent to it - in one of the four directions (up, down, left, or right), if it has the same value of the current cell.
Also, you cannot move to the cell that you visited in your last move. For example, the cycle (1, 1) -> (1, 2) -> (1, 1) is invalid because from (1, 2) we visited (1, 1) which was the last visited cell.
Return true if any cycle of the same value exists in the grid, otherwise, return false.

Let's denote the following variable,

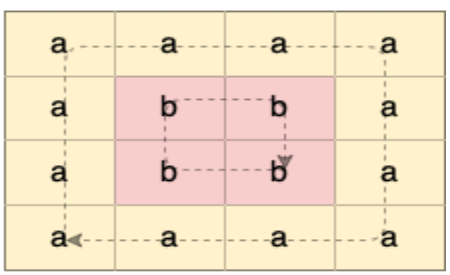
m → Number of rows in the grid

n → Number of columns in the grid

Example 1:

a	a	a	a
a	b	b	a
a	b	b	a
a	a	a	a

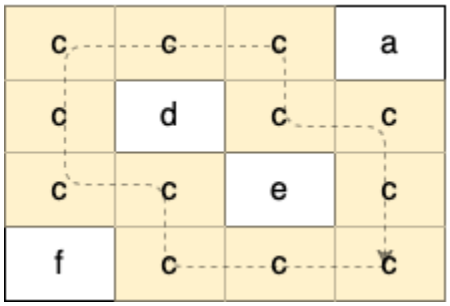
Input: grid = [["a","a","a","a"],["a","b","b","a"],["a","b","b","a"],["a","a","a","a"]]
Output: true
Explanation: There are two valid cycles shown in different colors in the image below:



Example 2:

c	c	c	a
c	d	c	c
c	c	e	c
f	c	c	c

Input: grid = [["c","c","c","a"],["c","d","c","c"],["c","c","e","c"],["f","c","c","c"]]
Output: true
Explanation: There is only one valid cycle highlighted in the image below:



Example 3:

a	b	b
b	z	b
b	b	a

Input: grid = [["a","b","b"],["b","z","b"],["b","b","a"]]
Output: false

Constraints:

m == grid.length
n == grid[i].length
1 <= m, n <= 500
grid consists only of lowercase English letters.

Test case 1:
Enter number of rows: 3
Enter number of columns: 3
Enter the grid values:
a a a a a b a a b
true
Test case 2:
Enter number of rows: 4
Enter number of columns: 4
Enter the grid values:
a a a a a b c a a b c a a a a a
true
Test case 3:
Enter number of rows: 3

Enter number of columns: 3
Enter the grid values:
a b c d e f g h i
false

Test case 4:

Enter number of rows: 2
Enter number of columns: 2
Enter the grid values:
a a a a
false

Test case 5:

Enter number of rows: 2
Enter number of columns: 3
Enter the grid values:
a a a a b a
true

Test case 6:

Enter number of rows: 4
Enter number of columns: 5
Enter the grid values:
a a a a a a b a a a a b a a a a a a a a
true

Test case 7:

Enter number of rows: 4
Enter number of columns: 4
Enter the grid values:
a b c d b c d a c d a b d a b c
false

Test case 8:

Enter number of rows: 1
Enter number of columns: 1
Enter the grid values:
a
false

Test case 9:

Enter number of rows: 4
Enter number of columns: 3
Enter the grid values:
a a a a b a a b a a a a
true

Test case 10:
Enter number of rows: 5
Enter number of columns: 5
Enter the grid values:
a b c d e f g h i j k l m n o p q r s t u v w x y
false

10. An n x n matrix is valid if every row and every column contains all the integers from 1 to n (inclusive).
Given an n x n integer matrix matrix, return true if the matrix is valid. Otherwise, return false.

Let's denote the following variable,

n → Number of rows or column in the square matrix

Example 1:

1	2	3
3	1	2
2	3	1

Input: matrix = [[1,2,3],[3,1,2],[2,3,1]]
Output: true
Explanation: In this case, n = 3, and every row and column contains the numbers 1, 2, and 3.
Hence, we return true.

Example 2:

1	1	1
1	2	3
1	2	3

Input: matrix = [[1,1,1],[1,2,3],[1,2,3]]
Output: false
Explanation: In this case, n = 3, but the first row and the first column do not contain the numbers 2 or 3.
Hence, we return false.

Constraints:

n == matrix.length == matrix[i].length
1 <= n <= 100
1 <= matrix[i][j] <= n
n should be a square matrix

Test case 1:
Input:
matrix =[[1, 2],[2, 1]]
Output: true
Explanation: Every row and column contains the numbers 1 and 2.
Test case 2:
Input
matrix = [[1, 2, 3, 4], [2, 3, 4, 1],[3, 4, 1, 2], [4, 1, 2, 3]]
Output: true
Explanation: Every row and column contains the numbers 1, 2, 3, and 4.
Test case 3:
Input:
matrix = [[1, 2, 3],[3, 1, 2], [2, 3, 1]]
Output: true
Explanation: Every row and column contains the numbers 1, 2, and 3.
Test case 4:
Input:
matrix = [[-1, -2, -3],[-3, -1, -2],[-2, -3, -1]]
Output: false
Test case 5:
Input:
matrix = [[1, 2, 3],[3, 'a', 2],[2, 3, 1]]
Output: false
Test case 6:
Input:
matrix = [[1, 2],[1, 2]]
Output: false
Explanation: The first row and the second row do not contain the number 1 and 2.
Test case 7:
Input:
matrix = [[4, 1, 3, 2],[2, 4, 1, 3], [3, 2, 4, 1],[1, 3, 2, 4]]
Output: true
Explanation: Every row and column contains the numbers 1, 2, 3, and 4.

Test case 8:
Input:
matrix = [[1, 2, 3, 4],[4, 3, 2, 1],[2, 1, 4, 3], [3, 4, 1, 2]]
Output: true
Explanation: Every row and column contains the numbers 1, 2, 3, and 4.
Test case 9:
Input:
matrix = [[1, 2, 3], [2, 3, 3], [3, 1, 2]]
Output: false
Explanation: The second row contains duplicate 3s and does not contain the number 1.
Test case 10:
Input:
matrix = [[1, 2, 4],[4, 1, 2],[2, 4, 1]]
Output: false
Explanation: The numbers should be from 1 to 3, but there is a 4 in the matrix.
Test case 11:
Input:
matrix = [[5, 1, 4],[1, 4, 5],[4, 5, 1]]
Output: false
Explanation: The numbers should be from 1 to 3, but there is a 5 in the matrix.